

# What I need from you before the fix run

**Companion to:** docs/planning/SECURITY-AUDIT.md (85 findings, consolidated from two independent audits)

**Written:** 12 August 2026

**Purpose:** every decision only you can make, and every action only you can take, in one place.

## How to use this document

There are three kinds of thing in here:

| Marker | Meaning                                                                                                                                                   |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| D#     | A <b>decision</b> . I explain it from the ground up, give you the options and what each one costs you, and tell you what I'd choose. You write an answer. |
| A#     | An <b>action</b> . Something only you can do because it needs a password or a credit card. Step by step, click by click.                                  |
| M#     | A <b>mid-run input</b> . Something I'll need <i>while</i> I'm working, not before. Listed so nothing surprises you.                                       |

Work through Parts 1–5 (decisions) first — they're reading and thinking, roughly 60–90 minutes. Then Part 6 (actions) — that's clicking, roughly 2–3 hours spread over a couple of sittings, and some of it involves spending money.

**You do not need to do them in order**, except where I say so explicitly in Part 6 (three of the actions have to happen before I touch certain code, and one has to happen after).

At the end there's an **answer sheet** (Part 9) — a compact form. Fill that in, and Part 8 is the message you paste into a fresh session to start the work.

## First: my honest read on your plan

You said: I answer everything, you do everything uninterrupted, app goes to 100.

**The plan is right. Three things about it are not going to work as stated, and you should know now rather than discover it in the middle.**

### 1. This is not one session's work. It's roughly six to ten.

The audit has 85 findings. The Critical and High ones — the stuff that genuinely must not ship — are about a week of focused work. The full list, including the compliance layer, disaster recovery, the scalability rewrites, the test suite, and the schema migration, is **two to three months**. Not because any single item is hard, but because there are 85 of them and a good number touch the database or the UI, and your own rules in CLAUDE.md require screenshots at two viewports, two rounds of comparison, and a full npm run check after every change. That's the right standard. It's also slow by design.

What actually happens: a session fills up its memory and has to stop. So instead of "one long run", the shape is **a written phase plan that every session reads first, works the next phase, logs what it did, and stops**. You start the next session with one sentence. I'll write that phase plan as the first thing after you've answered this document — it becomes the spine of the whole project.

## 2. "Release soon" and "solve everything" are in tension, and you should pick.

If you have a date, we split the 85 into *launch-blocking* and *scheduled*. My proposed line: **everything Critical, everything High, the new trust model, and bot protection must ship before a single real member signs up**. That's about 2 weeks including the trust model and the UI work. Everything else — the privacy layer excepted, see D14 — is genuinely fine to do in the weeks after launch, because it protects against scale and process failures rather than against an attacker.

If you don't have a fixed date, we just work the list top to bottom and launch when the High band is closed. **See D20.**

**3. Your input isn't only at the beginning.** There are seven points where I'll be blocked without you (Part 7). The two big ones: after I delete the admin backdoor you'll need to set a real password before you can get back into your own admin panel, and I can't write the privacy policy without knowing who legally owns this thing.

**One sequencing point that matters more than anything else in this document.** Two of the actions in Part 6 must happen **before** I start writing code:

- **A4 (create a separate development database)** — because production and your laptop currently share one database. The fix run involves schema changes. I don't want to be running those against live member data.
- **A1 (take a manual database backup)** — same reason.

*(A2 — protecting the photo archive — used to be the third and most urgent item here. It's done: Cloudflare turned out not to offer versioning at all, so I set up the fallback the item described instead — a nightly copy of every photo to the private backup bucket that never propagates deletions. Nothing for you to do.)*

Do those two this week even if you read nothing else.

---

# Part 1 — Decisions: the trust model

---

This is the part you raised, and it's the biggest design decision in the project, so I'm going to build it up from the beginning. It's also the fix for two findings I added to the audit after you raised it (**H21** and **H22**).

## Where things stand today

---

Your app currently has **two separate ideas of "is this person real"**, and they do very different amounts of work.

**Signal one: the confirmed email address.** When someone signs up, they get a link in their inbox and click it. That flips a flag. This is automatic and instant — nobody has to approve anything. The code calls this `emailConfirmed`.

Today this signal is doing all the work. It controls whether you can post, comment, upload a photo, contribute to the Collection, and whether you can see other people's phone numbers and email addresses.

**Signal two: the verified badge.** That's the little leaf next to someone's name. You grant it by hand from the admin panel. The code calls this `verifyState`, and it has four possible values: `unverified` (everyone starts here), `pending`, `verified`, `flagged`.

**Today this signal does nothing at all.** I checked every place in the code that reads it: it draws the leaf, and it fills your admin queue. **No permission anywhere depends on it.** It's decoration.

**So the practical situation is this.** The strong signal — you, a human, confirming this person actually went to Rishi Valley — controls nothing. The weak signal — proving you can open an email inbox, which any bot does in about four seconds with a throwaway address — controls everything.

And on top of that, the front door barely closes. The signup gate is two trivia questions whose answers are in any article about the school, with fuzzy matching so near-misses pass. There's no CAPTCHA anywhere on the site. And separately (finding H1), the member directory can currently be read **without logging in at all** because of a missing check.

You already sensed this. You're right.

## What you asked for

---

Until a person is both email verified and profile verified, they can't do anything destructive on the app such as post or anything else. They shouldn't be able to ruin stuff. We have to protect this from scrapers and spam bots. They should also not be allowed to see sensitive information that shows in get in touch.

That's a **two-gate model**, and it's the right instinct. Everything below is me turning that instinct into something precise enough to build. Four decisions.

---

### D1 — How does someone actually become "profile verified"?

**What this is about, from the ground up.**

If verification stops being decoration and starts being the thing that unlocks the app, then **whatever grants it becomes the bottleneck for your entire launch.** Right now, the only way to get verified is: you personally open the admin panel and click a button on that person's row.

Do the arithmetic. If 400 alumni sign up in your first week — plausible for a school with a strong network and a WhatsApp announcement — that's 400 rows, each needing you to decide "do I believe this person is who they say they are?" Even at a brisk 20 seconds each with a good interface, that's over two hours of

clicking, and it arrives in bursts. Meanwhile, every one of those 400 people is sitting in an app that won't let them do anything, wondering if it's broken.

That's not a security problem, it's a launch problem — but it's caused by a security decision, so you should make it deliberately.

### **Your options.**

**Option A — Manual only.** You verify everyone by hand, as today.

- *Good:* maximum control. Nobody gets in that you didn't personally wave through. Zero build cost.
- *Bad:* you are the bottleneck. Slow launch, frustrated new members, and a queue that grows while you're asleep. If you're travelling for a week, the app effectively stops accepting new members.

**Option B — Match against the school's alumni list ("roster match").** You get a list from the school office — names, batch years, and ideally admission numbers. When someone signs up, the app checks their details against that list. Match → verified instantly, no human involved. No match → falls into your manual queue.

- *Good:* the overwhelming majority of real alumni are verified **instantly and automatically**. Your queue only contains genuine edge cases — married surnames, nicknames, people the office missed, teachers. It's also a *stronger* check than your judgement, because it's the school's own record.
- *Bad:* you have to obtain the list, and it has to be reasonably accurate. There's also a privacy angle: you'd be holding a copy of the school's alumni roster, which is itself personal data and needs to be covered by whatever the school agrees to (see D15).
- *Build cost:* about a day, including a matching routine that's forgiving about spelling and surname changes.

**Option C — Vouching.** Two already-verified members vouch for a new person and they're in.

- *Good:* scales by itself, feels like a community, handles the long tail that no list will ever cover.
- *Bad:* it only works once you already have a decent base of verified members, so it can't carry your launch. And it's gameable if two accounts collude — though for a school alumni network, honestly, that risk is small.
- *Build cost:* about a day and a half, including the UI.

**Option D — Hybrid: roster match, falling back to your manual queue, with vouching added later.**

- Everything Option B gives you, plus a defined path for the people it misses, plus a pressure valve you can switch on after launch when there are enough verified members for it to work.

**What I'd do: Option D**, built in that order — roster matching first, manual queue always available, vouching switched on a month after launch if the manual queue is still annoying you.

**But it all hinges on one thing I can't find out for you:** does a usable alumni list exist, and can you get it?

**Please answer both parts:**

### D1a — Can you get an alumni list from the school office?

- ☐ Yes, and I can get it in a usable file (Excel/CSV/Google Sheet)
- ☐ Yes, but it'll be a PDF / printed / messy — *(that's fine, tell me and I'll write something to clean it up)*
- ☐ Maybe, I need to ask — *(please ask before the fix run starts; it changes the build)*
- ☐ No / not realistic

**If yes: what columns does it have?** (e.g. name, batch year, year left, admission number, house)

Answer: \_\_\_\_\_

**Roughly how many names?** \_\_\_\_\_

### D1b — Which model do you want?

- ☐ A — manual only ☐ B — roster only ☐ C — vouching ☐ **D — hybrid (recommended)**

Notes: \_\_\_\_\_

---

## D2 — What can someone do at each stage?

### What this is about.

Once there are two gates, there are three kinds of account, and each needs its own set of permissions:

- **Stage 0** — just signed up, hasn't clicked the email link yet.
- **Stage 1** — email confirmed, waiting on profile verification.
- **Stage 2** — email confirmed *and* profile verified. A full member.

Stage 2 is easy: everything. Stage 0 is easy: essentially nothing. **Stage 1 is the real decision**, and it's a genuine trade-off with no free answer:

- If Stage 1 can see **a lot**, then a spam bot that gets past the CAPTCHA and confirms a throwaway email address can still scrape most of your community's data. Verification protects nothing that matters.
- If Stage 1 can see **almost nothing**, you're properly protected — but a real alumnus who just signed up at 11pm stares at an empty app until you wake up and click a button. That's a bad first impression and some of them won't come back.

Here's my proposed split. **Read the Stage 1 column and tell me if you want it looser or tighter.**

| What they're doing                                          | Stage 0 | Stage 1        | Stage 2 |
|-------------------------------------------------------------|---------|----------------|---------|
| Sign in, see and edit their <b>own</b> profile and settings | ✓       | ✓              | ✓       |
| See their own contact details                               | ✓       | ✓              | ✓       |
| Message you (admin) for help                                | ✓       | ✓              | ✓       |
| <b>Read</b> the feed                                        | ✗       | ✓ read-only    | ✓       |
| <b>Read</b> the directory: names, batch, photo              | ✗       | ✓              | ✓       |
| <b>Read</b> the directory: city, job title, employer        | ✗       | ✗              | ✓       |
| See someone's full profile page                             | ✗       | partial        | ✓       |
| <b>See contact details (Get in touch)</b>                   | ✗       | ✗ locked panel | ✓       |
| Post, comment, like, vote in polls                          | ✗       | ✗              | ✓       |
| Upload any image                                            | ✗       | ✗              | ✓       |
| Contribute to the Collection                                | ✗       | ✗              | ✓       |
| Create or join a Catch-up                                   | ✗       | ✗              | ✓       |
| Report someone or something                                 | ✗       | ✗              | ✓       |
| Write a Letter                                              | ✗       | ✗              | ✓       |
| Ask to be verified                                          | —       | ✓              | —       |

**The reasoning behind the Stage 1 row, so you can argue with it.** I've drawn the line at **what a school yearbook already contains**. Name, batch and face are, for practical purposes, already semi-public for a school cohort — a scraper getting those has gained very little. Whereas **city, employer, job title, phone and email** are exactly the combination that makes targeted phishing work ("Hi, it's Ravi from ICSE '04, I saw you're at Deloitte in Bangalore now..."). Those stay behind the human check.

It also means a new member sees a living, populated app while they wait — real names, real faces, a real feed — which makes waiting feel like waiting rather than like a broken product.

### Two alternatives if you disagree:

- **Tighter:** Stage 1 sees nothing but a "we're checking your details" screen. Maximum protection, harshest onboarding. Only sane if D1 gives near-instant verification.
- **Looser:** Stage 1 reads everything except contact details. Nicest onboarding, but then verification only protects contact details and the ability to post — the whole directory is scrapeable by anyone with a throwaway inbox.

### D2 — Your answer:

☐ **The table above as written (recommended)**

☐ Tighter — Stage 1 sees nothing until verified

☐ Looser — Stage 1 can read everything except contact details

☐ The table but with changes:

Changes: \_\_\_\_\_

---

## D3 — What happens to the people already signed up?

### What this is about.

There are already real people in your database. The moment I switch this gate on, **every one of them whose `verifyState` isn't already `verified` loses the ability to post.** If they're mid-conversation in a Catch-up, it stops working for them with no warning.

**First, a number I need from you.** Open your app, go to `/admin`, and read off:

- total members: \_\_\_\_\_
- how many are **not** yet verified: \_\_\_\_\_

*(The admin page shows both. I deliberately haven't queried your production database myself — it's live member data and there's a fix in this project specifically about the fact that developers can currently touch it. I'd rather you read it off the screen.)*

### Your options.

**Option A — Grandfather everyone.** Everyone who exists on switch-on day is marked verified automatically. Clean slate, nobody is disrupted, and the gate applies only to new signups from that day.

- *Good:* zero disruption, zero support burden.
- *Bad:* if any junk accounts are already in there, they get promoted to full members permanently. **This is only safe if you'd personally vouch for the current list.**

**Option B — Verify nobody; everyone re-qualifies.** The gate applies to everyone. Existing members drop to Stage 1 until you (or the roster match) verify them.

- *Good:* the trust model actually means something from day one.
- *Bad:* you'll get messages from confused existing members. If the number above is large, that's a real support load.

**Option C — Grandfather selectively.** Auto-verify anyone who's been reasonably active (has posted, or has a completed profile, or signed up before a date you choose); everyone else re-qualifies.

- *Good:* the middle path — real users are undisturbed, dormant or suspicious accounts get checked.
- *Bad:* you need to pick the criteria.

**What I'd do:** if the current membership is small enough that you could scroll it and recognise most names — **Option A**, plus you skim the list first and block anything that looks wrong. It's the least disruptive and, at this size, your eyes are a better filter than any rule. If it's large or you don't recognise the names, **Option C** with "signed up before today AND has either posted or filled in their batch".

**D3 — Your answer:** ☐ A — grandfather everyone ☐ B — everyone re-qualifies ☐ C — selective (criteria: \_\_\_\_\_)

Members total: \_\_\_\_\_ · Not yet verified: \_\_\_\_\_

---

## D4 — The locked "Get in touch" panel

### What this is about.

You described it well: someone clicks *Get in touch* and instead of the contact details, a lovely blurred panel appears explaining they need to be verified. I want to confirm three details, because one of them is a security detail that's easy to get wrong.

**The security detail, and it's important.** There's a real trap here, and your codebase already has a comment about it (in `src/lib/email-verification.ts`): *"a phone number withheld by CSS is a phone number sitting in the page source."*

Meaning: if we fetch the real phone number, send it to the browser, and then blur it with a visual effect — the number is still **in the page**. Anyone can hit Ctrl+U, or open developer tools, and read it. The blur would be theatre.

So the way I'll build it: **the real details never leave the server for an unverified viewer**. What gets blurred is a set of fake placeholder shapes — grey bars roughly the length a phone number would be — purely so the panel looks like it's covering something. The effect is identical to look at, and the data genuinely isn't there. Your existing code already does the right thing here (`maySeeContacts` decides what to *serialize*, not just what to *render*); I'll extend the same approach.

### The three details I need from you:

**(a) The copy.** Two different situations need two different messages:

- *Haven't confirmed their email yet:*

#### Confirm your email to see this

We sent a link to `you@example.com`. Click it and contact details unlock straight away.

[ Resend the link ]

- *Email confirmed, waiting on verification:*

#### Verified members only

Contact details are kept for people we've confirmed are part of the school. We're checking your details now — you'll get a notification the moment you're in.

[ How verification works ]

**(b) Does this apply to your own profile?** My assumption: **no** — you can always see your own details. (Otherwise unverified members can't check what they entered.)

**(c) What about the vCard download and the map?** The profile page also offers a downloadable contact card and shows people's cities on a map. My assumption: **the vCard follows the same rule** (it's the same data in a file), and **the map stays visible at city level** for Stage 1, since a city is much coarser

than a street address and it's a big part of what makes the app feel alive. Tell me if you'd rather lock the map too.

#### D4 — Your answer:

Copy: ☐ use mine as written ☐ use mine with changes ☐ I'll write my own — draft: \_\_\_\_\_

Own profile always visible to yourself: ☐ yes (recommended) ☐ no

vCard follows the same lock: ☐ yes (recommended) ☐ no

Map visible at Stage 1: ☐ yes (recommended) ☐ no, lock it too

## D5 — How you'll know someone is waiting, and how fast you'll answer

### What this is about.

If verification gates the app, then someone waiting on you is someone who can't use the thing. Today there is **no signal at all** that anyone is waiting — nothing writes the `pending` state, and nothing tells you. You'd have to remember to check the admin page.

**What I'll build regardless:** a member-facing "request verification" button (which finally makes the `pending` state mean something), and a count on your admin page.

### What I need you to choose: how you get told.

- **Option A — Email digest.** One email a day at a time you pick, listing who's waiting. Quiet, batched, easy to ignore for a day.
- **Option B — Email per request.** Immediate, but during launch week this could be a hundred emails.
- **Option C — Nothing; you'll check the admin page.** Zero noise, and entirely dependent on you remembering.

**What I'd do:** A, with a threshold — the daily digest, *plus* an immediate email if the queue goes over 25, so a launch surge doesn't sit unnoticed.

**And one honest question:** what turnaround can you actually promise? Whatever you say, I'll put it in the waiting screen, because "we usually verify within a day" is enormously more reassuring than silence. Don't be optimistic — say what's true on a bad week.

**D5 — Your answer:** ☐ A — daily digest + surge alert (recommended) ☐ B — one email per request ☐ C — nothing

Digest time (your time zone): \_\_\_\_\_

Turnaround to promise on-screen: ☐ within a few hours ☐ within a day ☐ within 2–3 days ☐ don't state one

# Part 2 — Decisions: your account, and getting back in

---

## D6 — You need a real password, and there's a gap where you might lock yourself out

---

### What this is about, from the ground up.

Right now there are two ways to become the administrator of your site, and neither needs a password.

1. Type your email address into the normal login form with *any* password — even leaving it blank — and you're in as admin. The code checks whether the email matches yours and returns "yes, admin" *before* it ever looks at the password.
2. Send one line of text to a public web address on your own site ( `/api/auth/admin-login` ) containing just your email address, and it hands back a 30-day administrator pass. No password, no limit, no logging.

And **your email address is printed inside the JavaScript that gets sent to every visitor's browser**. I confirmed this by searching the actual built files, not by guessing. So anyone who opens your site, views the page source, searches for an `@`, and sends that one line, becomes the administrator of your alumni community. It takes under a minute and needs no tools beyond a web browser.

This is the single worst finding in the audit and both independent reviews put it first.

**The fix is to delete both paths.** After that, you log in like everyone else: email plus a real password.

**Here's the gap you need to be aware of.** Your account may not currently *have* a working password — you've been using the bypass. So the moment I delete it, you could be locked out of your own admin panel.

### The safe order, which I'll follow:

1. **Before** I delete anything, you set a real password using the normal "forgot password" flow (this already works — it's the one genuinely excellent piece of security code in the app). That's **A9** in Part 6.
2. You confirm to me that you can sign in with it.
3. *Then* I delete both bypasses.

**A second thing to decide while we're here: you are the only administrator.** If you lose access to that email account, nobody can administer the community. There's no second admin, no recovery path, no break-glass procedure. I'd strongly suggest a second admin account — a spouse, a co-organiser, or just a second address you control — so a lost phone isn't the end of the project.

### D6 — Your answers:

Second admin account: ☐ yes — email: \_\_\_\_\_ ☐ no, I accept the risk

Do you want the old `/api/auth/admin-login` route kept for local development only (never on the live site, and behind a long random secret)?

☐ yes, it's convenient ☐ **no, delete it entirely (recommended — it's simpler and there's nothing to leak)**

---

## D7 — Two-factor authentication for admin

---

**What this is about.** Two-factor means that after your password, you also type a 6-digit code from an app on your phone (Google Authenticator, 1Password, Authy). It means a stolen password alone isn't enough.

Right now there's no two-factor anywhere, for anyone, including you. Given the admin account can read every private message and permanently delete any member, a stolen admin password is total compromise.

### Options:

- **A — Two-factor for admin accounts only.** You (and any second admin) set it up once. Ordinary members don't have it. About half a day to build.
- **B — Optional for everyone.** Members can turn it on if they want. Nicer, more work, and almost nobody will use it.
- **C — None for now.**

**What I'd do:** A. It's the one account where the consequences justify the small friction, and it's cheap to build.

---

**D7 — Your answer:** ☐ A — admins only (recommended) ☐ B — optional for everyone ☐ C — none for now

---

## D8 — How long people stay signed in

---

**What this is about.** At the moment, signing in gives you a pass that lasts **30 days**, and there is currently **no way to cancel one**. Not by signing out, not by changing your password, not by blocking the person. If someone's laptop is stolen, that pass keeps working for up to 30 days and nothing you do stops it.

I'm fixing the "no way to cancel" part regardless — that's a database change that lets a password reset, a block, or an account deletion instantly kill every existing session. That fix is the same mechanism that makes blocking someone actually work (finding H4), so it's happening either way.

The separate question is **how long the pass should last:**

- **30 days (today).** Most convenient — members almost never see a login screen. Longest exposure if a device is lost.
- **7 days.** The common choice for a community site. You'd sign in about once a week.
- **7 days for members, 1 day for admins.** Admin sessions are the dangerous ones; that's where the short timer earns its keep.

**What I'd do:** the third option. Members barely notice; the account that can delete everything re-authenticates daily.

**D8 — Your answer:** ☐ 30 days for all ☐ 7 days for all ☐ **7 days members / 1 day admin (recommended)**

## D9 — The authentication library is a beta, and it can't be fixed

**What this is about, plainly.**

Your app uses an off-the-shelf component called `next-auth` to handle signing in. You're on version `5.0.0-beta.30` — a **beta**, meaning the makers have not declared it finished.

There's a published security advisory against it, and it's a nasty one for your specific app: under certain misconfigurations, the check "is this person logged in?" can **fail open** — it says yes when it should say no. Your app asks that exact question at the top of all 86 of its actions. It's the load-bearing check in the entire system.

**Here's what I found when I went to fix it.** The obvious advice is "get off the beta and onto a stable release." **There is no stable version 5.** I checked the package registry directly: the newest release marked stable is version `4.24.15`, and version 5 exists only as betas, currently `beta.32`.

Worse, the advisory covers versions `4.24.8 through 5.0.0-beta.31` — which *includes* that stable `4.24.15`. So "going stable" would mean rewriting your entire login system to an older, different interface **and still being vulnerable**.

`5.0.0-beta.32` **is the only published version that isn't affected.** So the upgrade is `beta.30` → `beta.32`, and you launch on a beta authentication library. That's not a compromise I'm choosing; it's the only option that exists short of replacing the login system entirely.

**Your options:**

- **A — Upgrade to `beta.32` and launch.** Closes the advisory. You're on a beta, which is genuinely common for this library (a very large number of production Next.js sites are), but it means occasional breaking changes when you upgrade.
- **B — Upgrade to `beta.32` now, and plan a move to a different login system in 3–6 months.** Candidates: Better Auth (free, self-hosted), Clerk or Auth0 (paid services, roughly \$25–100/month at your size, but they handle two-factor, session management and audit logging for you). This is a 1–2 week project.

- **C — Move now, before launch.** Delays you by 1–2 weeks.

**What I'd do: B.** Upgrade today because it's free and closes a Critical finding; revisit properly once you have real members and know what you actually need. Moving auth systems before launch delays you for a benefit you can get later at the same cost.

**D9 — Your answer:** ☐ A — beta.32, done ☐ **B — beta.32 now, plan a move later (recommended)** ☐ C — move now, accept the delay

---

## Part 3 — Decisions: the bill

---

Several fixes need paid services. **None of these are optional in the sense that the finding goes away without them** — you can't back up a database you haven't paid to have backed up. But you can choose the level. Here's the whole picture, honestly.

## D10 — Monthly running costs

| Service                          | What it's for                                                                                                                                                                                                           | Free tier            | What you'd need     | Cost           |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|---------------------|----------------|
| <b>Supabase</b> (database)       | <b>Backups.</b> The free tier has no meaningful backup. If your database is corrupted or wrongly deleted, everything is gone. The Pro plan gives daily backups and point-in-time recovery.                              | not sufficient       | Pro                 | ~\$25/mo       |
| <b>Supabase — second project</b> | A <b>separate database for development</b> , so my work and your laptop never touch live member data. This is the single biggest ongoing-risk reduction in the whole audit.                                             | sufficient           | Free                | \$0            |
| <b>Resend</b> (email)            | Sending confirmation and password-reset emails. Free is capped at ~100/day, which your app already has an entire priority-queue system built to work around, and which an attacker can exhaust in minutes (finding M3). | too small for launch | Pro                 | ~\$20/mo       |
| <b>Upstash Redis</b>             | The memory that rate limiting needs — "this address has tried to log in 40 times in a minute, stop it." Without a shared store, limits reset constantly and don't work.                                                 | likely sufficient    | Free → paid if busy | \$0—10/mo      |
| <b>Sentry</b> (error tracking)   | Tells you when the app breaks for a real person. Right now, <b>nothing</b> tells you — a scheduled job on your site has been failing every night for weeks and nobody noticed.                                          | sufficient           | Free                | \$0            |
| <b>Cloudflare Turnstile</b>      | The CAPTCHA that stops bulk bot signups.                                                                                                                                                                                | sufficient           | Free                | \$0            |
| <b>Cloudflare R2</b>             | Photo storage. A nightly copy of every photo to the private backup bucket (so deleted photos are recoverable) costs a little extra storage.                                                                             | mostly sufficient    | —                   | ~\$1–3/mo      |
| <b>Vercel</b> (hosting)          | Worth checking which plan you're on. The Hobby plan doesn't permit commercial use and limits scheduled jobs to one a day.                                                                                               | depends              | Pro if needed       | \$0 or \$20/mo |

**Total: roughly \$45–80 per month, most likely ~\$65.**

### D10 — Your answer:

☐ Approve all of it (~\$65/mo)

☐ Approve, but skip: \_\_\_\_\_ (tell me which, and I'll write down which finding stays open as a result)

☐ I need to think — my ceiling is \$\_\_\_\_\_/mo

**Which Vercel plan are you on?** ☐ Hobby ☐ Pro ☐ not sure

## D11 — Should member photos stay publicly readable?

---

### What this is about, from the ground up.

Every photo in your app — profile pictures, post images, the whole Collection archive — sits at a web address that **anyone in the world can open if they know the link, forever**. No login required. That's how it's set up today.

Practically, this means: if someone deletes their account, their photos stay reachable at their old links permanently. If a link is ever shared or ends up in a search engine, it stays live. And a "removed" Collection photo isn't removed — the app only hides it from the page; the file is still there.

### Options:

- **A — Leave as is.** Fastest, cheapest, and honestly what most small community sites do. But it means you cannot truthfully say "delete my account and my photos are gone", which is something people can legally require.
- **B — Signed links.** Photos are private, and the app generates a temporary link (valid for, say, an hour) each time it shows one. Deletion becomes real, and stolen links expire. Costs a bit more complexity and a small amount of speed, since links can't be cached as aggressively.
- **C — Middle path.** Profile photos and the Collection stay public (they're the least sensitive, and the Collection is arguably a heritage archive people want shared). Anything a member uploads privately goes behind signed links.

**What I'd do: B**, in the post-launch phase rather than before launch. It's the honest answer to "delete my data" and to a private community. But it's a couple of days' work and it isn't what an attacker exploits first, so it shouldn't hold up your release.

**D11 — Your answer:** ☐ A — leave public ☒ **B — signed links, after launch (recommended)** ☐ B but before launch ☐ C — middle path

---

## Part 4 — Decisions: legal, privacy, and who owns this

---

I'm not a lawyer and this isn't legal advice. But the audit scored compliance at **12 out of 100**, and the reason is simple: there is currently **no privacy policy, no cookie notice, no record of consent, and no stated basis for collecting anyone's data at all**. For a site holding people's phone numbers, employers, home cities and photographs, that's a bare violation of European data law with no defence available — and your alumni body certainly includes people in the EU and UK.

Most of this is a first-time-setup job, not a technical one. I can draft everything. **I need facts from you.**

## D12 — Who legally owns and runs this?

---

**Why this matters.** Every privacy policy has to name a **data controller** — the person or organisation legally responsible for the data. It's whoever decides what gets collected and why. That name goes in the policy, on any agreements with your service providers, and it's who a regulator or an unhappy member would contact.

### Your options:

- **A — You personally.** Simplest. Your name and a contact address go in the policy. Legally the responsibility is yours personally.
- **B — Rishi Valley school / the KFI.** Only correct if the school has actually agreed to run this and take responsibility. It also means their legal people should review it.
- **C — An alumni association.** If a registered body exists, it's the natural home.
- **D — A company you set up for it.** Overkill unless this becomes something bigger.

**What I'd do:** A to launch, unless the school has formally adopted the project. It's honest, it's accurate, and you can transfer it later. You'll need a contact address in the policy — you can use a dedicated address like `privacy@rishivalley.space` rather than your personal one.

### D12 — Your answers:

Controller: ☐ A — me personally ☐ B — the school ☐ C — an association ☐ D — a company

Exact legal name to print: \_\_\_\_\_

Contact address for privacy questions:

\_\_\_\_\_

Postal address (a policy needs one; a city and country is usually enough):

\_\_\_\_\_

---

## D13 — What is the school's actual relationship to this?

---

**Why I'm asking.** Two reasons, and the second one might surprise you.

First: if you're getting the alumni list (D1), the school is handing you personal data about thousands of people. That needs to be a deliberate act on their side, ideally in writing, however informal.

Second, and this is the subtle one: **Rishi Valley is a Krishnamurti school.** Under European data law, information that reveals someone's *religious or philosophical beliefs* is "special category" data with much stricter rules. A reasonable argument exists that "attended a Krishnamurti Foundation school" is exactly that. I'm not saying it definitely is — this is genuinely arguable and a lawyer should say. But it's the kind of thing that's cheap to handle now (clear notice, explicit consent at signup) and expensive to handle after a complaint.

**Practically:** I'll write the signup consent to be explicit and specific regardless. It costs nothing and covers the argument.

### D13 — Your answers:

School's position: ☐ formally supports it ☐ informally aware and fine with it ☐ doesn't know yet ☐ not involved

Anything in writing from them? ☐ yes ☐ no

Do you want me to flag the special-category question in the policy for a lawyer to review later? ☐ yes ☐ no

## D14 — What we keep, and for how long

**What this is about.** At the moment, **nothing is ever deleted**. Notifications, reports, private messages, sent-email records with people's addresses in them — all of it accumulates forever. Law in most places says you keep personal data only as long as you actually need it, and that you say so up front.

Here's what I propose. These are ordinary, defensible numbers — override any of them.

| Data                                                         | Proposed                                     | Why                                    |
|--------------------------------------------------------------|----------------------------------------------|----------------------------------------|
| Account and profile                                          | Until they delete it                         | Obvious                                |
| Posts, comments, photos                                      | Until deleted, or account deletion           | Obvious                                |
| Private messages to admin                                    | 2 years after the conversation closes        | Long enough to reference a past issue  |
| Reports and moderation notes                                 | 3 years                                      | Safeguarding matters may resurface     |
| Sent-email log (addresses + subjects)                        | 90 days                                      | Only useful for debugging delivery     |
| Notifications                                                | 1 year                                       | Nobody reads a year-old notification   |
| Login and admin audit log ( <i>new — doesn't exist yet</i> ) | 1 year                                       | Long enough to investigate an incident |
| Payment/contribution records                                 | 7 years                                      | Tax rules generally require this       |
| Deleted accounts                                             | Purged within 30 days, including photographs | Grace period for accidental deletion   |

**One item deserves a specific decision: the 30-day grace period on deletion.** Today, deleting your account is instant, irreversible, unconfirmed and silent — and if you've ever filed a report it crashes and doesn't delete at all. I'm fixing all of that. The question is whether "delete" should mean *immediately and forever*, or *deactivate now, purge in 30 days, with an email saying so*. The grace period is much kinder to people who click in anger, and it's standard practice.

### D14 — Your answers:

Retention table: ☐ approve as-is ☐ approve with changes: \_\_\_\_\_

Account deletion: ☐ **30-day grace period then purge (recommended)** ☐ immediate and permanent

## D15 — Consent at signup

**What this is about.** Right now, someone signs up and their data is stored with no notice and no record that they agreed to anything. What's needed is a checkbox — an *unticked* one, ticked deliberately — plus a record of when they ticked it.

**What I'll add to the signup form:**

☐ I agree to the [Privacy Policy](#) and [Community Guidelines](#), and I understand my name, batch and profile will be visible to other verified members of this community.

And in the database: the date they agreed and which version of the policy they agreed to.

**Two things I need from you:**

**(a) Community Guidelines.** You don't have any. For a private alumni community you want a short page — what this space is for, what isn't acceptable, what happens if someone crosses the line. I'll draft it in your voice from the existing site copy; **you'll need to approve it** (that's **M4** in Part 7).

**(b) Existing members haven't agreed to anything.** Options: show them the notice once at their next sign-in and record their agreement, or email everyone. The in-app prompt is far less annoying and works better.

### D15 — Your answers:

Signup consent checkbox: ☐ yes, as drafted ☐ yes, with changes: \_\_\_\_\_

Community Guidelines: ☐ you draft, I'll approve ☐ I'll write them ☐ skip for now

Existing members: ☐ **one-time prompt at next sign-in (recommended)** ☐ email everyone ☐ skip

## Part 5 — Decisions: loose ends

### D16 — The `/lab` rooms are public on the live site

**What this is about.** You have 38 development and preview pages under `/lab`. They're reachable by anyone on the live site — no login. `CLAUDE.md` says "remove before shipping to the public"; that hasn't happened.

One of them, `/lab/everything`, publishes an internal findings log **containing exact file paths and quoted source code from your app**. That's a free map of the building for anyone attacking it.

**Options:** ☐ A — **admin only** (you keep them, nobody else sees them) · ☐ B — remove from the live site entirely, keep locally · ☐ C — leave public

**What I'd do:** A. You keep everything you built and lose nothing; the door just closes.

**D16 — Your answer:** ☐ A (recommended) ☐ B ☐ C

---

## D17 — Bot protection on the front door

**What this is about.** I want to add **Cloudflare Turnstile** to signup, login and password reset. It's Cloudflare's alternative to Google's reCAPTCHA: usually invisible, no picture puzzles, doesn't track people, and free. You're already on Cloudflare for photo storage, so it's the natural fit.

This is what actually stops bulk bot signups, and it works alongside the trust model rather than replacing it.

The only cost is a small amount of friction for real people, and very occasionally a checkbox.

**D17 — Your answer:** ☐ **yes, add Turnstile (recommended)** ☐ no ☐ yes, but only on signup, not login

---

## D18 — The public demo site

**What this is about.** You run a separate no-login demo on its own hosting and database. The audit found its protection is genuinely well built — a default-deny system that blocks writes at the database layer, and it's the one piece of security code with a test.

**But it shares this codebase.** So as I change the trust model, the demo's invented visitor needs to keep working, or the demo turns into a wall of "please verify your email" messages on a site whose whole job is to look finished.

**What I'll do:** keep the demo exempt from the new gates, exactly as it's already exempt from the email gate, and re-run its test after every change. **I just need to know it's still live and still matters to you.**

**D18 — Your answer:** ☐ yes, keep it working ☐ it's retired, ignore it ☐ retire it as part of this work

---

## D19 — Publicly shown contribution totals

---

**What this is about.** There's a small unresolved item in the audit (L8). Your payment system correctly distinguishes **real** payments from **test** ones. If any page shows a public total — "₹X raised so far" — it must count only the real ones, or your test payments inflate the number.

I couldn't find the query that produces a public total, which may mean it doesn't exist.

**D19 — Your answer:** Do you show a public total anywhere? ☐ yes — where: \_\_\_\_\_ ☐ no ☐ not sure, please check

## D20 — Your launch date, and where the line goes

---

**What this is about.** As I said at the top: 85 findings is two to three months of work. If you're launching sooner, we need a line between "must be done before a real member signs up" and "scheduled for after."

**My proposed launch-blocking set** (about **2–3 weeks** of work):

- All 4 Critical findings
- All 22 High findings
- The two-gate trust model (Part 1)
- Bot protection (D17)
- Privacy policy, consent, guidelines (Part 4) — *legally you can't take a member's data without these*
- Rate limiting everywhere
- Error tracking, so you find out when it breaks

**Scheduled for after launch** (the following 4–8 weeks): the schema rewrite, signed photo links, the test suite, the scalability work, field encryption, data export, and the code-quality items.

**D20 — Your answer:**

Target launch date: \_\_\_\_\_ ☐ no fixed date, launch when the High band is closed

Line: ☐ **as proposed above (recommended)** ☐ different — move these into "before launch":

\_\_\_\_\_

## Part 6 — Actions: step by step

---

Everything below needs your hands, because it needs a password or a card.

**Do A1 and A4 first, before I start writing code.** The rest can happen while I work. (A2 is already done — see below.)

**A10 (rotate the secrets) must be done LAST** — after the backdoor is closed. Rotating keys while the front door is open achieves nothing.

## A1 — Take a manual database backup right now 🕒 10 min • DO FIRST

**Why.** You have one database serving both the live site and your laptop, and there's no confirmed backup. Everything else in this project involves changing that database. I want a snapshot on your own disk before anything happens.

1. Go to <https://supabase.com/dashboard> and sign in.
2. Click your project (the one for the live site, region **Mumbai / ap-south-1**).
3. In the left sidebar find **Database**, then look for **Backups**.
4. **Read what it says and write it down for me** — specifically whether automated backups exist, how often, and how far back. On the free plan you'll likely see either nothing or an upsell. *This answer matters: it tells us whether finding C4 is "no backups" or "backups nobody has ever tested".*
5. Now take a manual one regardless. Still in the sidebar, find **Database** → **Backups** or **Settings** → **Database**, and look for a **Download / Backup** option. If there isn't one on your plan, tell me and **I will write you a one-command export script instead** — that's a five-minute job for me and it doesn't need dashboard access.
6. Save the file somewhere that is **not** your laptop's main drive — an external disk or cloud storage.

**Report back:** what the Backups page says: \_\_\_\_\_ · manual backup saved: ☐  
yes ☐ no, needs a script

## A2 — Protect the photo archive from deletion 🕒 done • nothing for you to do

**Why this existed.** Any member can currently delete **every photograph in your app** — every profile picture, every Collection scan, every post image — with a single ordinary request. Until this item, that deletion would have been permanent and unrecoverable.

**What changed.** This item originally asked you to turn on "object versioning" in the Cloudflare dashboard. You looked, and it isn't there — because Cloudflare R2 doesn't offer versioning at all, on any plan. That's not you missing a setting; the feature doesn't exist. So I built the fallback this item already named: every night, the same automated job that backs up the database now also copies every photo to the private backup bucket, and **deletions are never copied across** — a photo deleted from the live bucket, by accident or by attack, stays safe in the backup until we deliberately remove it.

The one gap: a photo uploaded and deleted within the same day, before the nightly copy has seen it. The code fix for the deletion bug itself (C2) closes that.

**Report back:** nothing — this one's done. You'll see it as the "media" job in the nightly backup run on GitHub.

### A3 — Check your Supabase and Vercel plans 🕒 5 min

1. **Supabase** → your project → **Settings** → **Billing** (or the plan badge near the project name). Write down the plan.
2. **Vercel** → <https://vercel.com/dashboard> → your account/team settings → **Billing**. Write down the plan.

**Report back:** Supabase plan: \_\_\_\_\_ · Vercel plan: \_\_\_\_\_

### A4 — Create a separate development database 🕒 20 min · DO FIRST

**Why.** Today, one database serves the live site *and* your laptop *and* mine. Every test I run writes to real member data. Your own `CLAUDE.md` documents a near-miss where a routine command would have dropped live tables — and the protection against it is a written note asking a human to be careful. That's not a safeguard; that's a hope.

This is the single highest-value ongoing change in the entire audit, and the free tier is enough.

1. <https://supabase.com/dashboard> → **New project**.
2. Name it `rv-connect-dev`. Same region (**Mumbai / ap-south-1**) so behaviour matches. Generate a strong database password and **save it in your password manager immediately** — Supabase won't show it again.
3. Wait for it to finish setting up (2–3 minutes).
4. Go to **Settings** → **Database** (or **Connect**) and copy **two** connection strings:
  - the **transaction pooler** one (port **6543**)
  - the **session pooler / direct** one (port **5432**)
5. Send both to me. **These are secrets — put them somewhere private, not in a chat that's backed up publicly.** If in doubt, paste them straight into your local `.env` file and just tell me "done", and I'll pick them up from there.

I'll then copy the structure across and load realistic fake data, so development never touches a real person's record again.

**Report back:** ☐ created · project name: \_\_\_\_\_ · connection strings: ☐ in my `.env` ☐ sent to you

## A5 — Set up rate limiting storage (Upstash) ⌚ 10 min

**Why.** To stop someone guessing passwords, the app has to remember "this address has tried 40 times in the last minute." It has nowhere to remember that right now — so every limit resets constantly and none of them work. Upstash is a small shared memory for exactly this. Free tier is fine.

1. Go to <https://upstash.com> → **Sign up** (you can use your GitHub account).
2. **Create Database** → choose **Redis**.
3. Name: `rv-connect-ratelimit`. Region: pick the one closest to **Mumbai / ap-south-1**. Type: **Regional** (not Global — cheaper and enough).
4. Once created, open it and find the section labelled **REST API**.
5. Copy these two values:
  - `UPSTASH_REDIS_REST_URL`
  - `UPSTASH_REDIS_REST_TOKEN`
6. Add both to Vercel (see **A8** for exactly how) **and** to your local `.env` file, in this exact format:

```
UPSTASH_REDIS_REST_URL=https://your-value-here.upstash.io
UPSTASH_REDIS_REST_TOKEN=your-token-here
```

**Report back:** ☐ done, both values added to Vercel and `.env`

## A6 — Set up error tracking (Sentry) ⌚ 10 min

**Why.** Nothing currently tells you when the app breaks for a real person. The proof: a scheduled job on your site has been failing every single night — it points at an address that doesn't exist — and nobody noticed. Sentry emails you when something breaks. Free tier is fine.

1. <https://sentry.io> → sign up.
2. **Create Project** → platform **Next.js** → name it `rv-connect`.
3. It'll show you a **DSN** — a long web address starting `https://` and containing `@sentry.io`. Copy it.
4. Add to Vercel and `.env`:

```
SENTRY_DSN=https://your-dsn-here@00.ingest.sentry.io/0
NEXT_PUBLIC_SENTRY_DSN=https://your-dsn-here@00.ingest.sentry.io/0
```

5. In Sentry: **Settings** → **Alerts**, and make sure alerts go to an address you actually read.

**Report back:** ☐ done · DSN added: ☐ Vercel ☐ `.env`

## A7 — Get bot-protection keys (Cloudflare Turnstile) ⌚ 5 min · *only if D17 = yes*

1. <https://dash.cloudflare.com> → left sidebar → **Turnstile**.

2. **Add site.** Name: `rv-connect`.
3. **Domains:** add **both** your live domain (e.g. `rishivalley.space`) **and** `localhost` — otherwise it breaks in development.
4. **Widget type:** choose **Managed** (Cloudflare decides when to challenge; usually invisible).
5. You'll get two values. Copy both:
  - **Site Key** (safe to be public)
  - **Secret Key** (must stay private)
6. Add to Vercel and `.env`:

```
NEXT_PUBLIC_TURNSTILE_SITE_KEY=your-site-key-here
TURNSTILE_SECRET_KEY=your-secret-key-here
```

**Report back:** ☐ done

## A8 — How to add an environment variable in Vercel 🕒 reference

You'll need this for A5, A6, A7 and A10. Here's the procedure once.

1. <https://vercel.com/dashboard> → click the **rv-connect** project.
2. **Settings** tab → **Environment Variables** in the left menu.
3. **Key** = the name (e.g. `UPSTASH_REDIS_REST_URL`), **Value** = the value.
4. **Environments:** tick **Production**, **Preview** and **Development** unless I say otherwise.
5. **Save.**
6. **Important:** environment variables only take effect on the *next* deployment. After adding them, go to the **Deployments** tab, find the most recent one, click the `⋮` menu and choose **Redeploy**.

**Also, while you're in there — one deletion.** Find `NEXT_PUBLIC_ADMIN_EMAIL` and **delete it**. That's the variable that puts your email address into every visitor's browser. Nothing will break: the code that reads it is being deleted in the same run, and the variable does nothing on its own.

**Report back:** ☐ I know how to do this • ☐ `NEXT_PUBLIC_ADMIN_EMAIL` deleted

## A9 — Set a real admin password 🕒 5 min • DO BEFORE I DELETE THE BYPASS

**Why.** You've been getting in without a password. When I delete that route you'll need a real one, and I want you to have confirmed it works *before* the door closes.

1. Go to your live site's `/login` page.
2. Click **Forgot password**.
3. Enter your admin email address. (*You'll get the same "if that address exists, we've sent a link" message either way — that's deliberate, so the form can't be used to find out who's a member.*)

4. Open your email, click the link.
5. Set a strong password. **Use your password manager to generate it** — 20+ characters, saved immediately. This password protects every member's private data.
6. **Now sign out completely and sign back in with it.** This is the step that matters — don't skip it.

**Tell me explicitly when you have signed in with a real password.** I will not delete the bypass until you have.

**Report back:** ☐ password set AND I have successfully signed in with it

## A10 — Rotate every secret 🕒 45 min • DO LAST — after the backdoor is closed

**Why.** The admin backdoor has been open on a live site. We have no way to know whether anyone found it, because there's no logging of any kind. The safe assumption is that everything the server knows may have leaked. That means every key gets replaced.

**Order matters:** do this **after** I've confirmed the backdoor is closed and deployed. Rotating keys while it's still open accomplishes nothing.

**Do them one at a time and verify the site still works after each.**

### A10.1 — The session signing key

1. Generate a new random value. On your Mac, open **Terminal** and run: `openssl rand -base64 32`
2. Copy the output.
3. Vercel → **Settings** → **Environment Variables** → find `NEXTAUTH_SECRET` (and `AUTH_SECRET` if it exists) → **Edit** → paste → **Save**.
4. Update your local `.env` with the same value.
5. Redeploy (A8, step 6).
6. **Everyone gets signed out, including you.** That's expected and correct — it's the point.

### A10.2 — The photo storage keys

1. Cloudflare → **R2** → **Manage R2 API Tokens**.
2. **Create API Token.** Permissions: **Object Read & Write**. Scope it to the `rv-alumni-media` bucket only.
3. Copy the **Access Key ID** and **Secret Access Key** — *shown once, so save immediately*.
4. Update in Vercel and `.env`: `R2_ACCESS_KEY_ID`, `R2_SECRET_ACCESS_KEY`.
5. Redeploy, then **upload a test photo to confirm it works**.
6. Only once that works: go back and **delete the old token**.

### A10.3 — The email key

1. <https://resend.com/api-keys> → **Create API Key** → permission **Sending access**.
2. Update `RESEND_API_KEY` in Vercel and `.env`. Redeploy.

3. Test by triggering a password reset to yourself.
4. Delete the old key.

#### A10.4 — The payment keys

1. <https://dashboard.razorpay.com> → **Settings** → **API Keys** → **Regenerate**.
2. Update `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET` and the webhook secret in Vercel and `.env`.
3. **Careful:** payments will fail between regenerating and redeploying. Do it at a quiet time.
4. Test a small real contribution end to end.

#### A10.5 — The database password

1. Supabase → **Settings** → **Database** → **Reset database password**.
2. Update `DATABASE_URL` and `DIRECT_URL` in Vercel and `.env` — the password appears inside both strings.
3. Redeploy. **The site is down between the reset and the redeploy**, so do this last and quickly.

**Report back:** ☐ session key ☐ R2 keys ☐ Resend ☐ Razorpay ☐ database — all rotated and verified

### A11 — Lock down the GitHub repository 🕒 15 min

**Why.** Right now, any push to `main` deploys straight to your live site with no check of any kind — no tests, no review, nothing. I'm going to add automated checks; this makes them mandatory rather than advisory.

1. Go to <https://github.com/sanan-shankar/rv-connect> → **Settings**.
2. **Branches** (left menu) → **Add branch protection rule** (or **Add rule**).
3. Branch name pattern: `main`
4. Tick **Require status checks to pass before merging**. *(The list will be empty until I've added the automated checks — come back and tick them once I tell you they're running. That's **M6** in Part 7.)*
5. Tick **Do not allow bypassing the above settings** — or leave it unticked if you want to be able to push directly in an emergency. Your call; unticked is more forgiving for a solo project.
6. Now **Settings** → **Code security and analysis**, and turn on:
  - **Dependabot alerts** — tells you when a component you use has a security hole
  - **Dependabot security updates** — opens the fix automatically
  - **Secret scanning** — shouts if a password is ever committed by accident
  - **Push protection** — blocks the commit before it happens
7. Check the repository is **Private: Settings** → **General** → scroll to the bottom → **Danger Zone**. If it says "Make public", you're private and fine.

**Report back:** ☐ branch rule added ☐ Dependabot on ☐ secret scanning on ☐ repo is private

## A12 — Verify your email sending domain 🕒 20 min (if not already done)

**Why.** If your confirmation and password-reset emails aren't properly authenticated, they land in spam — and since email confirmation is now gate one of two, an email in spam means a member who can't use the app at all.

1. <https://resend.com/domains> → check whether your domain is listed and **Verified**.
2. If not: **Add Domain**, enter it, and Resend gives you three DNS records (SPF, DKIM, DMARC).
3. Add them in **Cloudflare** → **your domain** → **DNS** → **Records**, copying each value exactly.
4. Back in Resend, click **Verify**. It can take up to an hour.

**Report back:** ☐ already verified ☐ just did it ☐ needs help

## A13 — Get the alumni list 🕒 depends on the school • *only if D1a = yes*

1. Ask the school office for the alumni roster.
2. Ideal: **CSV or Excel**, one row per person, with **name**, **batch year**, **year left**, and **admission number** if they have it.
3. A PDF or scan is workable — tell me and I'll write something to extract it.
4. **Put the file in docs/private/ in the project folder** (I'll add that folder to the ignore list so it can never be committed to GitHub) and tell me it's there.
5. **Do not** email it to yourself or put it in a shared drive — it's a few thousand people's personal data.

**Report back:** ☐ file at docs/private/\_\_\_\_\_ ☐ asked, waiting ☐ not doing this

## Part 7 — What I'll need from you during the run, and at the end

None of these can be done up front. Listed so nothing catches you out.

| #         | When                            | What I need                                                                                                                                                                              | Blocking?                 |
|-----------|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| <b>M1</b> | Before I delete the backdoor    | <b>A9 confirmed</b> — you've signed in with a real password                                                                                                                              | <b>Yes — hard stop</b>    |
| <b>M2</b> | Before every deployment         | Your approval to push. Pushing is deploying, and your rules say I never do that without asking. Expect 8–15 of these.                                                                    | <b>Yes, each time</b>     |
| <b>M3</b> | After the trust-model work      | Look at the locked "Get in touch" panel on your phone and desktop and approve the design. I'll send screenshots.                                                                         | Yes                       |
| <b>M4</b> | Mid-run                         | Approve the <b>privacy policy</b> and <b>community guidelines</b> I draft. Read them properly — they're statements you're making about your own conduct.                                 | Yes                       |
| <b>M5</b> | After A4 (dev database)         | Confirm the fake development data looks sensible                                                                                                                                         | No                        |
| <b>M6</b> | After I set up automated checks | Return to GitHub and tick the required checks (A11, step 4)                                                                                                                              | No                        |
| <b>M7</b> | Near the end                    | Sign the data-processing agreements with Vercel, Supabase, Cloudflare, Resend and Razorpay. These are usually a form in each dashboard; I'll give you the exact links when we get there. | Yes, before launch        |
| <b>M8</b> | Last                            | <b>A10 — rotate every secret</b>                                                                                                                                                         | <b>Yes, before launch</b> |
| <b>M9</b> | Last                            | Walk through the whole thing as a brand-new member on your phone: sign up, confirm, wait, get verified, post. Nothing substitutes for this.                                              | Yes                       |

## Part 8 — What to say in the new session

Once you've filled in Part 9 below, start a fresh session and paste this:

There's a security audit at `docs/planning/SECURITY-AUDIT.md` — 85 findings, 4 Critical, 22 High. I've answered every decision and completed the actions in `docs/planning/OWNER-INPUT-REQUIRED.md` — my answers are in Part 9 of that file.

Read both documents fully before doing anything. Then write the phase plan ( `docs/planning/SECURITY-FIX-PLAN.md` ) breaking the work into sessions, and start on phase one.

Rules: don't push without asking me. Don't touch the production database. Log everything in `progress.md`.

That's all it needs. The two documents carry the rest.

## Part 9 — Answer sheet

*Fill this in as you go. This is what the next session reads.*

=====

ANSWERS – filled in by Sanan on: \_\_\_\_\_

=====

#### TRUST MODEL

D1a Alumni list available? .....  
Columns: .....  
Roughly how many names: .....  
D1b Verification model (A/B/C/D): .....  
D2 Capability table (as-is / tighter / looser / changes):  
.....  
D3 Existing members (A/B/C): ..... Total: ..... Unverified: .....  
D4 Locked-panel copy: .....  
Own profile visible to self: ..... vCard locked: ..... Map at Stage 1: .....  
D5 Notifications (A/B/C): ..... Digest time: .....  
Turnaround to promise: .....

#### ACCOUNT & AUTH

D6 Second admin: ..... email: .....  
Delete the dev bypass route entirely: .....  
D7 Two-factor (A/B/C): .....  
D8 Session length: .....  
D9 next-auth plan (A/B/C): .....

#### MONEY

D10 Budget approved: ..... Skipping: .....  
Vercel plan: ..... Supabase plan: .....  
D11 Photo storage (A/B/C + timing): .....

#### LEGAL

D12 Controller: ..... Legal name: .....  
Privacy contact: ..... Postal: .....  
D13 School's position: ..... In writing: ..... Flag for lawyer: .....  
D14 Retention table: ..... Deletion grace period: .....  
D15 Consent checkbox: ..... Guidelines: ..... Existing members: .....

#### LOOSE ENDS

D16 /lab (A/B/C): .....  
D17 Turnstile: .....  
D18 Demo site: .....  
D19 Public contribution total shown anywhere: .....  
D20 Launch date: ..... Line as proposed: .....

-----

#### ACTIONS COMPLETED

-----

A1 Backup [ ] Backups page says: .....  
A2 Photo backup [x] (done for you -- R2 has no versioning; nightly copy set up instead)  
A3 Plans checked [ ]  
A4 Dev database [ ] Connection strings in .env: [ ]  
A5 Upstash [ ]  
A6 Sentry [ ]  
A7 Turnstile [ ]  
A8 NEXT\_PUBLIC\_ADMIN\_EMAIL deleted from Vercel [ ]  
A9 Real admin password set AND signed in with it [ ] <-- I cannot start C1 without this  
A10 Secrets rotated [ ] <-- LAST, after the backdoor is closed  
A11 GitHub locked [ ]  
A12 Email domain [ ]  
A13 Alumni list [ ] at: .....

-----  
ANYTHING ELSE I SHOULD KNOW  
-----  
.....  
.....

---

*If any question here doesn't make sense, don't guess — write "explain this again" next to it and I'll rewrite that section. A wrong answer to D1, D2 or D3 costs days of rework; a question you didn't understand costs one paragraph.*